Pre-entrega Módulo 6 — Agente de soporte ProTable con voz

Bruno Acorsi · Coderhouse, AI Automation Avanzado · 12/10/2026 · Workflow `checkpoint6_bruno_acorsi` (n8n, carpeta M6, publicado) · Telegram + OpenAI Whisper + ElevenLabs

1. Resumen y objetivo

El soporte de ProTable suma un **canal de voz por Telegram**. El encargado de un local manda una nota de voz con su consulta y recibe la respuesta **en audio**, en menos de 200 caracteres. El flujo hace tres cosas, en este orden: **escuchar** (Whisper transcribe la nota en español), **entender** (un AI Agent consulta el Manual de soporte de ProTable con la herramienta RAG del M5) y **responder con voz** (ElevenLabs genera el MP3 y Telegram lo envía).

Alcance. La voz es un canal **complementario** para consultas operativas cortas, las que pasan en medio del servicio, cuando el encargado tiene las manos ocupadas. El canal de mail del M4/M5 sigue siendo el principal para facturación y procesos largos.

Telegram Trigger → Get File (binario `data`) → OpenAI Whisper (`es`) → Switch de contingencia → Set: descarta el audio → AI Agent (gpt-4o-mini) ⇄ consultar_manual_protable → Code: límite 200 → ElevenLabs (MP3) → Telegram Send Audio → Set: fin sin binario

En violeta, lo que protege el flujo: el Switch filtra transcripciones inútiles y la herramienta del manual le da la fuente de verdad al agente. Cada falla (descarga, Whisper, ElevenLabs) tiene su salida de error con una respuesta en texto.

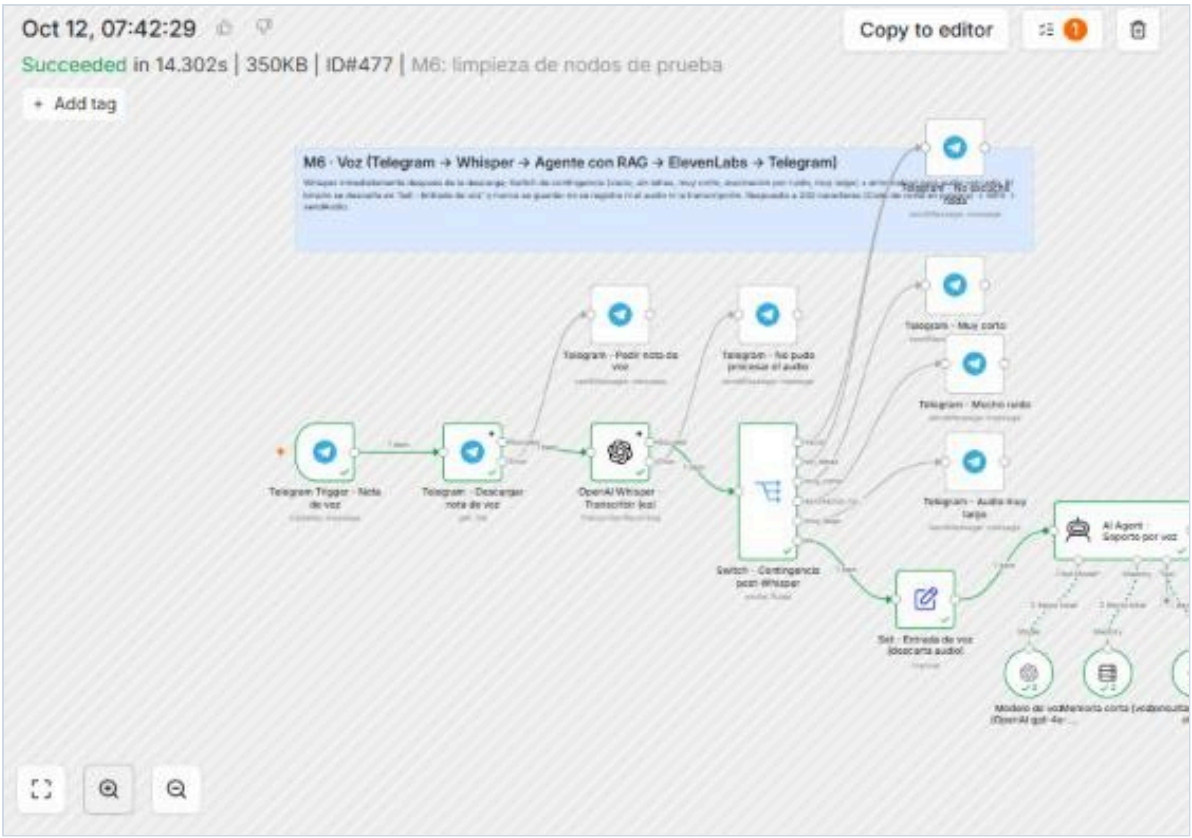
Qué se construyó y qué se reutilizó

Pieza	Detalle
`checkpoint6_bruno_acorsi`	Workflow de voz **propio**, publicado en la carpeta M6 de n8n. Canal: bot oficial de Telegram **@bacorsi_bot** (credencial `brunobot`). Una sticky note en el lienzo resume el flujo.
Herramienta reutilizada del M5	`consultar_manual_protable` → sub-workflow `rag_consultar_manual_modulo5_acorsi_bruno`: búsqueda híbrida sobre los 91 fragmentos del manual, **Top-K 4**, **Minimum Score 0,40**. Es la misma herramienta que usan los workers del M5: el manual tiene un solo punto de mantenimiento.
Proveedores	OpenAI (Whisper `whisper-1` y el modelo del agente `gpt-4o-mini`) · ElevenLabs (nodo verificado `@elevenlabs/n8n-nodes-elevenlabs`, credencial `Eleven_all`) · Telegram Bot API.

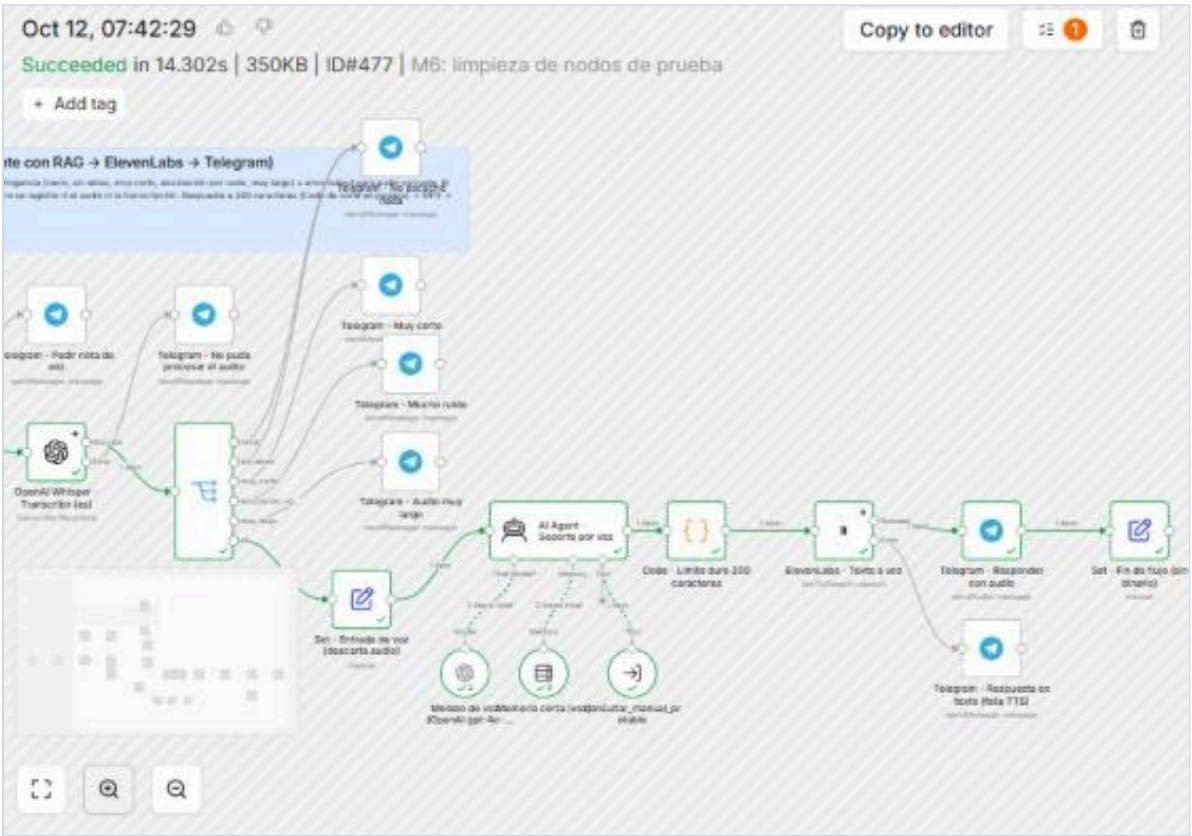
Decisión de diseño: un agente de voz propio, no un duplicado del M5

> El plan original era duplicar el manager del M5 y sumarle una segunda rama de entrada con los tres workers como herramientas. Se construyó otra cosa: un **AI Agent de voz propio** (Tools Agent) con **tres piezas**: el modelo **OpenAI gpt-4o-mini** con `maxTokens 200`, una **memoria corta por chat** (6 mensajes, clave `tg:<chat>`) y la **herramienta del manual**. Los workers **no** se conectaron como herramientas. Hay tres razones:
>
> - **Latencia.** Cada worker es otro agente con su propio modelo y su propia consulta al manual. El agente de voz llamaría a un agente que a su vez consulta el manual: dos rondas de modelo antes de empezar a generar el audio. La ejecución real de punta a punta (477) ya tardó **14,3 s** con una sola ronda. Quien espera una respuesta de voz en pleno servicio no tolera el doble.
> - **Simplicidad.** Las consultas de voz son operativas ("¿qué hago si…?") y las responde el manual. Los workers del M4/M5 están armados para el mail: devuelven JSON con fuentes, borrador HITL, CRM y Slack. Nada de eso sirve para una respuesta hablada de 200 caracteres.
> - **Un solo punto de control.** La regla de los 200 caracteres y la limpieza para voz se aplican a la salida de un único agente.
>
> Si más adelante hace falta registrar tickets por voz, el worker de Tech Support se conecta como herramienta (`toolWorkflow`) sin tocar el resto del flujo.

C0 · El lienzo con el recorrido real de la ejecución 477



C0a. Ejecución 477 (12/10, 07:42, *Succeeded in 14,302 s*), una nota de voz real por Telegram. En verde, el camino recorrido: Trigger → Descargar nota de voz → Whisper → Switch (salida `ok`) → Set - Entrada de voz → AI Agent con modelo, memoria y herramienta.

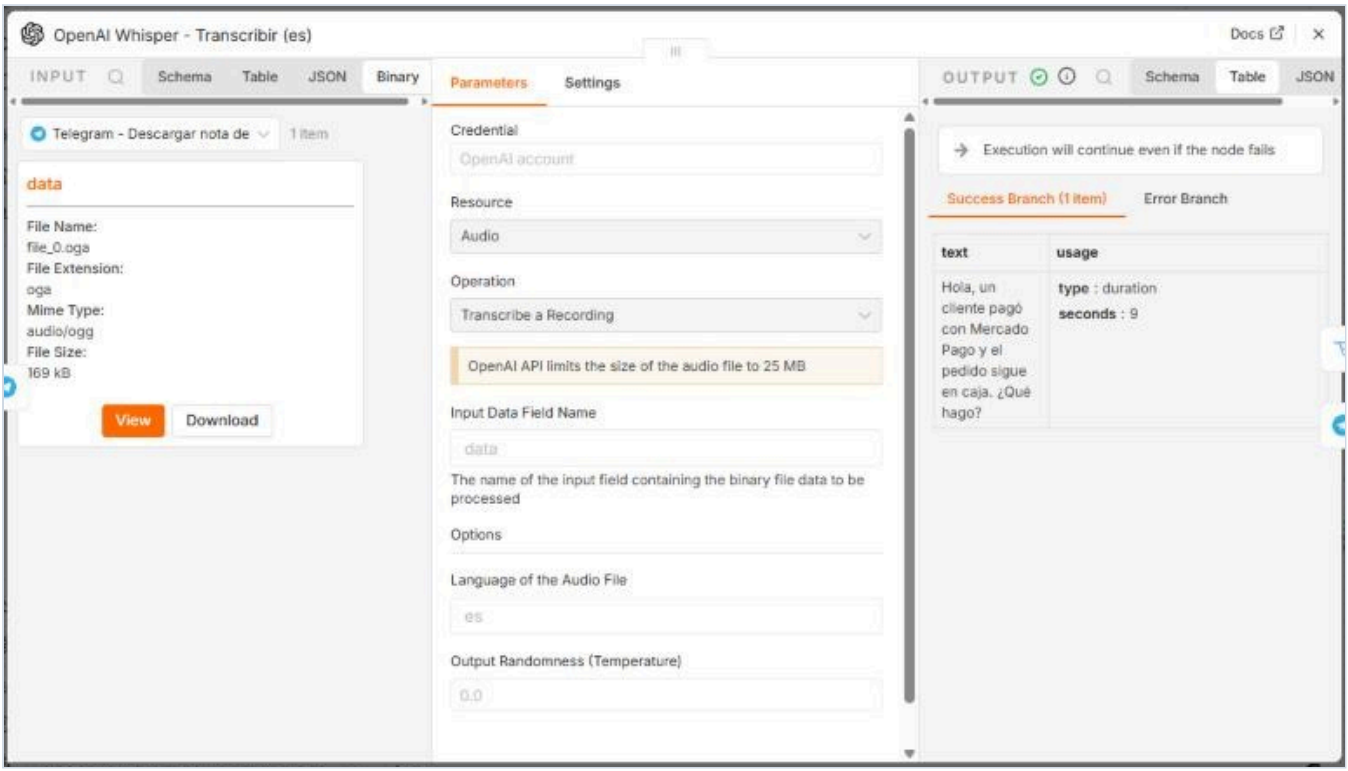


C0b. Continuación: AI Agent → Code - Límite duro 200 caracteres → ElevenLabs - Texto a voz → Telegram - Responder con audio → Set - Fin de flujo. Las salidas de contingencia (arriba) y el fallback *Respuesta en texto (falla TTS)* (abajo) no se usaron en esta ejecución.

Componente 1 · Trigger de notas de voz (Telegram)

El bot oficial **@bacorsi_bot** (creado con @BotFather) recibe los mensajes. El nodo `Telegram Trigger - Nota de voz` escucha el evento **Message**. El trigger de n8n no descarga notas de voz (solo foto, video y documento), así que el nodo siguiente, `Telegram - Descargar nota de voz`, toma el `file_id` de `message.voice` (o `message.audio`) y baja el archivo al binario `data`. Si el mensaje no trae audio (un texto, un sticker), la descarga falla por su **salida de error** y el bot contesta "*Hola, soy el soporte de ProTable. Mandame una nota de voz con tu consulta.*". Telegram no deja bajar archivos de más de 20 MB: un audio demasiado pesado también cae en esa salida.

Componente 2 · OpenAI Whisper, inmediatamente después de la descarga



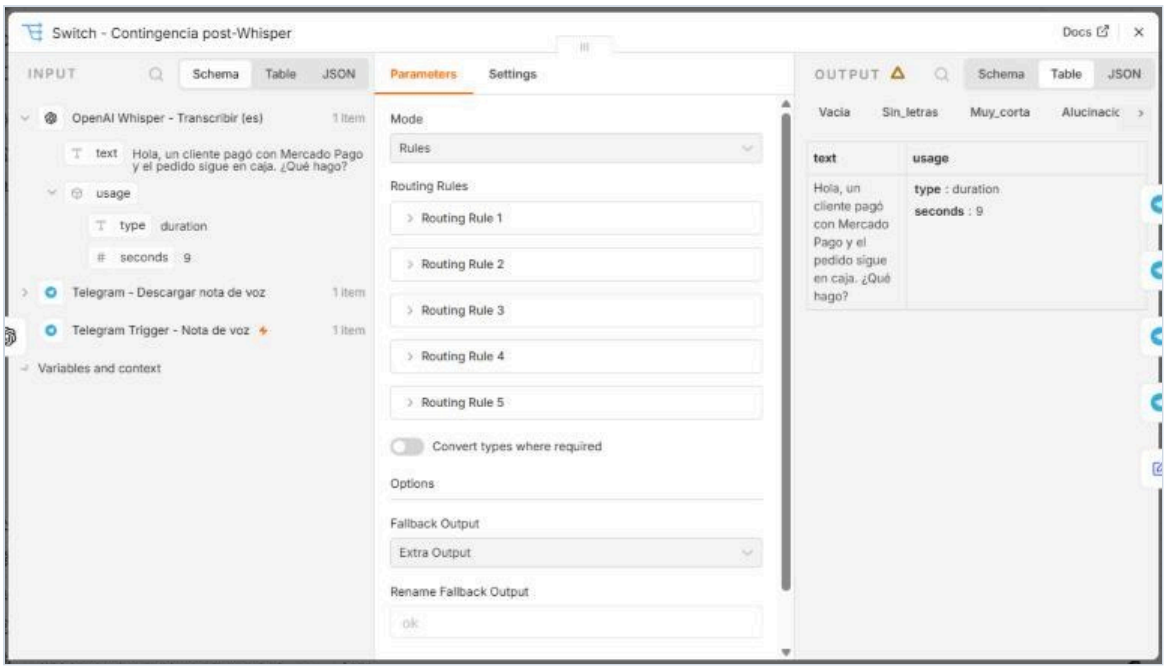
C2. Nodo `OpenAI Whisper - Transcribir (es)` en la ejecución 477. Entrada: binario `data`, `file_0.oga`, `audio/ogg`, 169 kB. Parámetros: Resource *Audio*, Operation *Transcribe a Recording*, **Input Data Field Name** = `data`, **Language** = `es` (cargado a mano), Temperature 0. Salida: el texto transcripto y 9 s de audio facturados.

Decisión	Por qué
Nodo nativo de OpenAI (`whisper-1`)	La consigna pide el nodo OpenAI Whisper con la propiedad binaria `data`. El `.oga` de Telegram entró sin renombrar: no hizo falta el Code de conversión del material de clase.
Idioma fijo en `es`	Evita que Whisper detecte mal el idioma en audios cortos o con ruido, y baja la latencia.
Temperature 0	Transcripción determinista: el mismo audio da el mismo texto.
On Error → salida de error	Un audio corrupto, un formato inválido o un problema del proveedor (sin saldo, timeout) no cortan el flujo: el usuario recibe "*No pude procesar tu audio. ¿Me lo mandás de nuevo o me lo escribís?*".

Entre el trigger y Whisper solo está la descarga del archivo, porque Telegram envía un `file_id` y no el audio.

Componente 7 · Contingencia inmediatamente después de Whisper

Va en este lugar del documento porque es el orden del flujo: Whisper → Switch → agente.



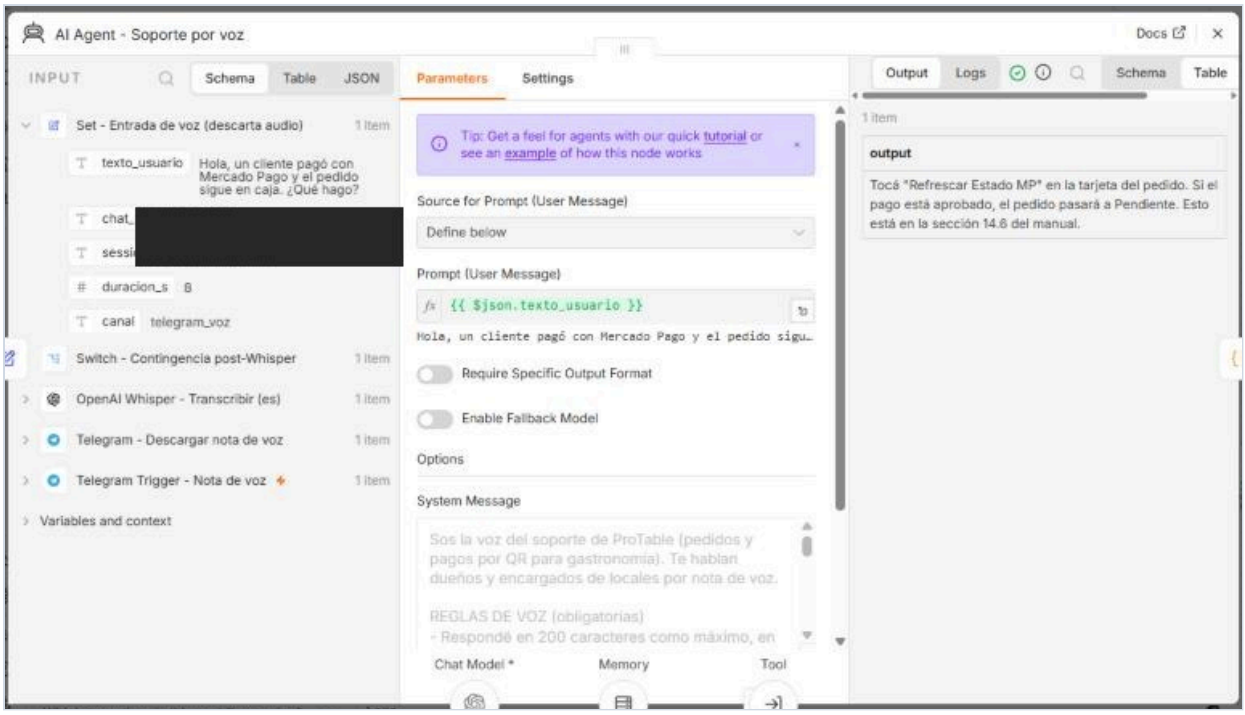
C7. Nodo `Switch - Contingencia post-Whisper`: modo *Rules*, 5 reglas, **Fallback Output = Extra Output** renombrada `ok`. Arriba, las salidas renombradas (Vacia, Sin_letras, Muy_corta, Alucinación…). En la ejecución 477 la transcripción salió por `ok`.

#	Salida	Condición	Qué detecta	Respuesta del bot (en texto)
E	*error de Whisper*	el nodo falló	audio corrupto, formato inválido, proveedor caído	"No pude procesar tu audio. ¿Me lo mandás de nuevo o me lo escribís?"
1	`vacia`	texto vacío	silencio total	"No escuché nada en el audio. Probá grabarlo de nuevo, cerca del micrófono."
2	`sin_letras`	sin ninguna letra	solo puntos, números o símbolos	
3	`muy_corta`	menos de 4 letras	"eh", "ok", un ruido suelto	"Te escuché muy poquito. ¿Me contás en una frase qué necesitás?"
4	`alucinacion_ruido`	frases típicas inventadas (Amara.org, "gracias por ver", "suscribete", "música de fondo")	Whisper "inventa" texto sobre silencio o ruido	"Había mucho ruido y no te entendí bien. ¿Me lo repetís en un lugar más tranquilo?"
5	`muy_largo`	más de 90 s o más de 1.200 caracteres	audio largo: costo y fatiga	"El audio es muy largo para este canal. Contame lo principal en menos de un minuto."
—	`ok`	ninguna de las anteriores	transcripción usable	→ `Set - Entrada de voz` → agente

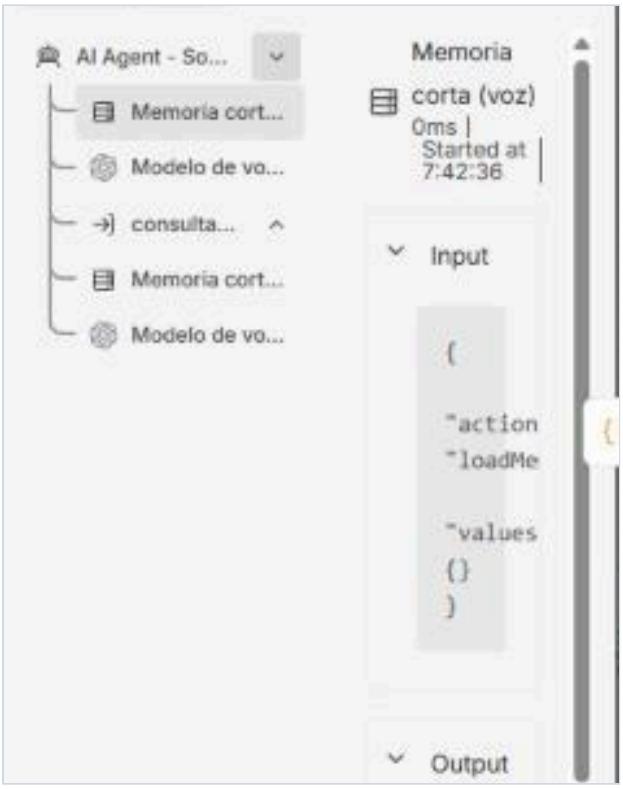
> **Evidencia clave (ejecución 461, banco de pruebas `M6_prueba_voz_sin_telegram`).** Se le pasaron a Whisper dos audios reales: una consulta generada con la voz Helena de Windows (8 s) y **3 s de silencio**. La consulta salió bien: "*¡Hola! Un cliente pagó con Mercado Pago y el pedido sigue trabado en caja. ¿Qué tengo que hacer?*". Sobre el silencio, Whisper **alucinó** "*Subtítulos realizados por la comunidad de Amara.org*". Un IF de largo mínimo (el del material de clase) lo habría dejado pasar, porque es una frase larga y con letras. El Switch lo mandó a la salida 4, `alucinacion_ruido`, y la consulta a `ok`.

Trade-off del corte por duración. La regla 5 va *después* de Whisper para respetar la consigna (Whisper inmediatamente después del audio). El costo: un audio de 3 minutos se transcribe igual y cuesta USD 0,018 (USD 0,006 por minuto) antes de ser rechazado. Se podría cortar antes de la descarga con `voice.duration`, pero eso aleja a Whisper del trigger. Los mensajes de contingencia van **en texto**: no se gasta TTS en errores.

Componente 3 · El texto transcripto llega al AI Agent central (Tools Agent)



C3. `AI Agent - Soporte por voz`. Entrada: el nodo `Set - Entrada de voz (descarta audio)` con `texto_usuario`, `session_id` (tapado), `duracion_s = 8` y `canal = telegram_voz`. **Prompt = Define below, `{{ $json.texto_usuario }}`**. Abajo, los tres puertos conectados: Chat Model, Memory y Tool. Salida del modelo, todavía con la cita de sección que después limpia el Code.



C3b. Pestaña *Logs* del agente en la misma ejecución: Memoria corta → Modelo de voz → **consultar_manual_protable** → Memoria corta → Modelo de voz.

En n8n, el AI Agent desde la versión 2 **es siempre un Tools Agent**: el modelo decide qué herramienta llamar y cuándo. Ya no hay selector de tipo de agente.

Los logs muestran el ciclo completo: el agente carga la memoria del chat, el modelo decide **consultar el manual**, recibe los fragmentos, guarda el turno en la memoria y el modelo redacta la respuesta final.

Sub-nodo	Configuración
Modelo	OpenAI `gpt-4o-mini`, temperature 0,3, **maxTokens 200** (techo de costo)
Memoria	Ventana de 6 mensajes por `session_id = tg:<chat>`. Guarda texto, nunca audio
Herramienta	`consultar_manual_protable` (sub-workflow RAG del M5, Top-K 4, Min 0,40)

Se usa el mismo modelo que los workers del M5, así la calidad de las respuestas es comparable entre el mail y la voz.

Componente 4 · Regla de 200 caracteres en el System Message

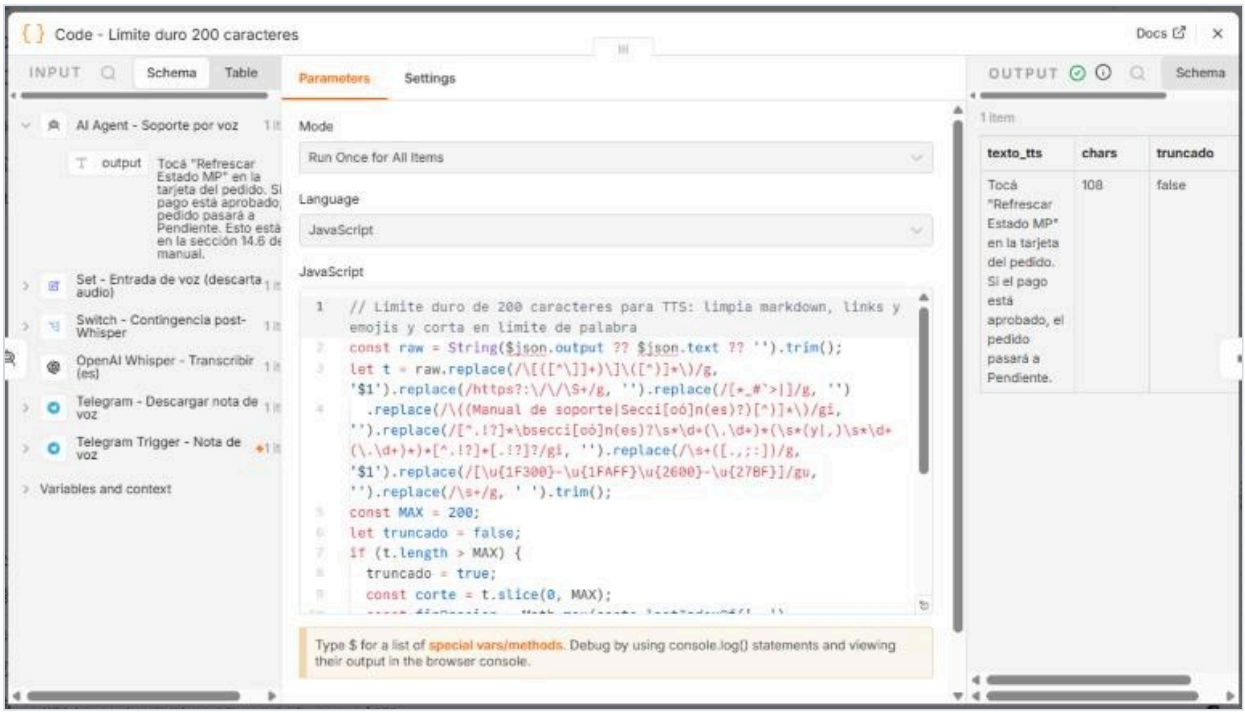
La regla vive en el System Message del agente. Las primeras líneas se ven en C3. Texto completo:

```
Sos la voz del soporte de ProTable (pedidos y pagos por QR para gastronomía). Te hablan dueños y encargados de locales por nota de voz.

REGLAS DE VOZ (obligatorias)
- Respondé en 200 caracteres como máximo, en una o dos oraciones cortas, porque tu respuesta se convierte en audio.
- Español rioplatense con voseo ("tocá", "revisá", "podés"). Sin listas, sin markdown, sin emojis, sin links. No nombres secciones ni el manual: en audio no sirven.
- No saludes ni te despidas: andá directo a la respuesta.

FUENTE DE VERDAD
- Para cualquier pregunta sobre ProTable, consultá SIEMPRE la herramienta consultar_manual_protable antes de responder, con el vocabulario del manual.
- Respondé solo con lo que dicen los fragmentos. Si no responden exactamente lo que te preguntan, decí: "No sé, eso no está en el manual. Te paso con una persona del equipo."
- Nunca prometas reembolsos, descuentos ni plazos. [...]

Si lo que llegó no se entiende o no es una consulta, pedí que lo repita en una frase.
```

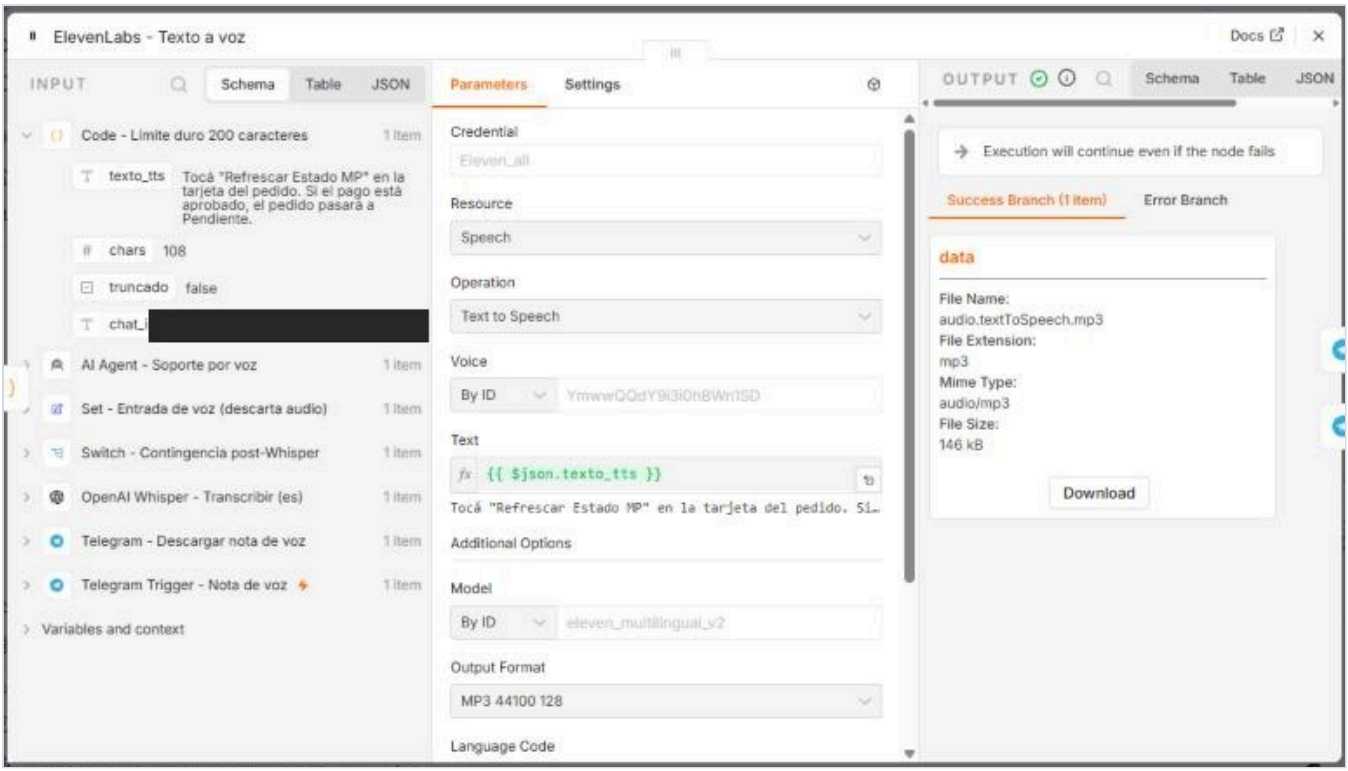


C4. `Code - Límite duro 200 caracteres` en la ejecución 477. Entrada: la salida del agente con *"Esto está en la sección 14.6 del manual."*. Salida: `texto_tts` limpio, **`chars: 108`**, **`truncado: false`**.

Regla en el prompt más control determinista. El modelo puede no cumplir; el Code no. Antes de ElevenLabs el Code: (1) quita links, markdown y emojis, que el TTS lee mal o deletrea; (2) quita las **citas de sección**, porque "sección 14.6" no le sirve a quien escucha; (3) si el texto supera 200 caracteres, corta en el último fin de oración o, si no hay uno razonable, en el último espacio, para que el audio nunca termine con una palabra cortada. Cada carácter de más se paga en ElevenLabs (USD 0,08 cada 1.000 a precio de lista de la API).

Por qué la limpieza de citas: en las ejecuciones 459 y 461 el agente agregaba "Esto está en la sección 14.6 del manual" pese a la instrucción del prompt. Se agregó la regla al Code y en la 475 la salida quedó en 114 caracteres limpios. En la 477 (C4) se ve el caso real: el modelo volvió a citar y el Code lo limpió.

Componente 5 · ElevenLabs con el nodo verificado (Multilingual, Stability y Clarity)

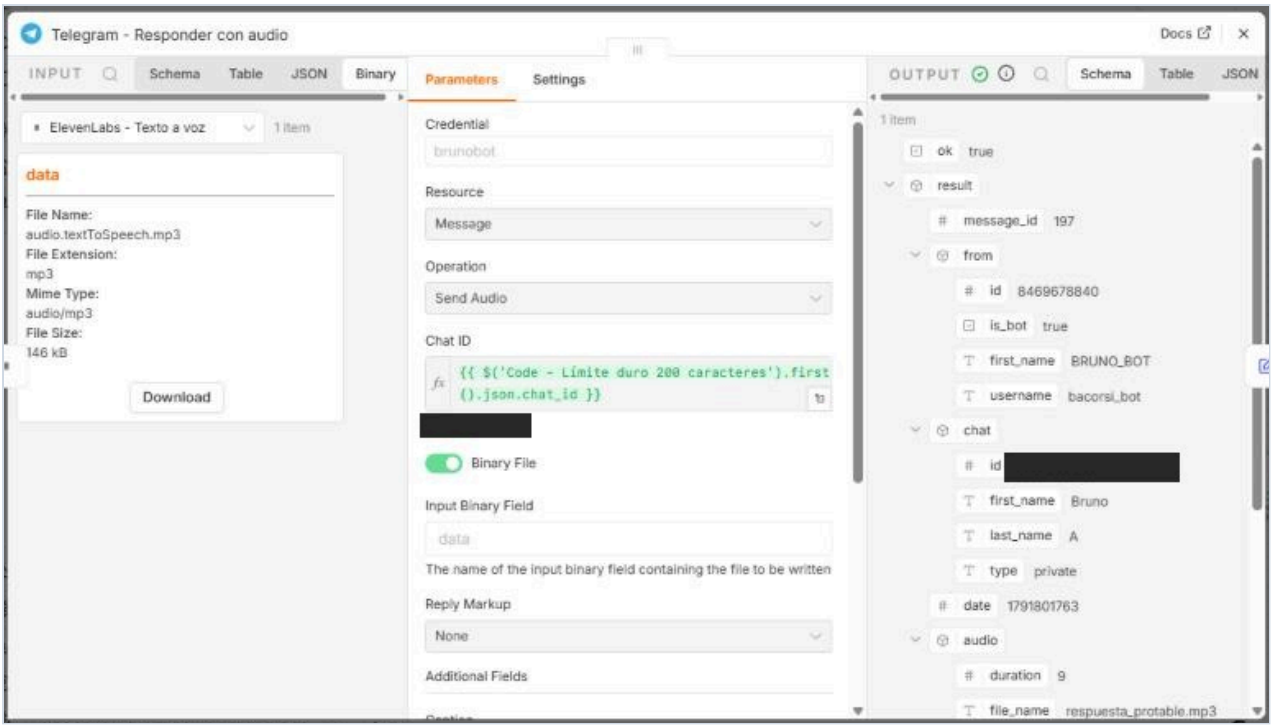


C5. `ElevenLabs - Texto a voz`, el nodo verificado `@elevenlabs/n8n-nodes-elevenlabs` (no un HTTP Request). Credential `Eleven_all`, Resource *Speech*, Operation *Text to Speech*, voz por ID, **Text = `{{ $json.texto_tts }}`**, **Model = `eleven_multilingual_v2`**, **Output Format = MP3 44100 128**. Salida: binario `data`, `audio.textToSpeech.mp3`, 146 kB.

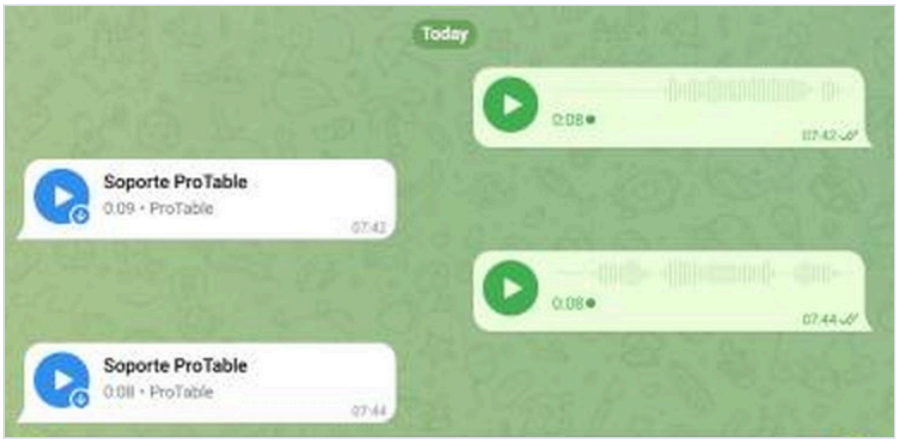
Parámetro	Valor	Justificación de marca
Voz	**Octavio – Calm Narrator** (por ID)	Voz calma y clara. Quien llama está en medio de un problema: una voz serena baja la tensión y se entiende en un salón o una cocina ruidosos. Siempre la misma voz: es la voz de la marca.
Modelo	`eleven_multilingual_v2`	El que pide la consigna. Mejor prosodia en español que los modelos Flash/Turbo, aunque cuesta el doble por carácter. El límite de 200 caracteres contiene ese costo.
Idioma	`es`	Fija el español para que la voz no cambie de acento si el texto trae una palabra en otro idioma ("Mercado Pago", "Refrescar Estado MP").
Stability	**0,55**	Por debajo de 0,4 la voz varía entre respuestas, y eso es inconsistencia de marca. Por encima de 0,75 suena monótona y robótica. 0,55 da calidez sin perder consistencia.
Similarity boost (Clarity)	**0,75**	Fiel a la voz elegida y con buena dicción. Por encima de 0,85 puede amplificar artefactos del audio de origen.
Formato	MP3 44100 Hz, 128 kbps	Es el formato que acepta Telegram Send Audio.

En el nodo de n8n no hay sliders: **Stability** y **Similarity boost** van en el campo JSON *Voice Settings* de las opciones adicionales (más abajo en el panel, fuera de C5): `{ "stability": 0.55, "similarity_boost": 0.75 }`. Si ese campo queda vacío, el nodo manda 1 y 1, y la voz suena robótica. **Fallback:** el nodo tiene *On Error → salida de error*. Si ElevenLabs falla (key inválida, cuota agotada, voz no permitida), la misma respuesta sale en texto por `Telegram - Respuesta en texto (falla TTS)`, visible en C0b. El cliente nunca se queda sin respuesta.

Componente 6 · Telegram Send Audio con el binario de ElevenLabs



C6. `Telegram - Responder con audio`: Operation **Send Audio**, **Binary File = on**, **Input Binary Field =** `data` (el MP3 de ElevenLabs, 146 kB), Chat ID por expresión desde el Code. Salida de Telegram: `ok: true`, `audio.duration: 9`, `file_name: respuesta_protable.mp3`.



C6b. Chat real con el bot @bacorsi_bot: la nota de voz del usuario y la respuesta en audio (MP3, Send Audio) para las ejecuciones 477 (07:42) y 479 (07:44). Las respuestas llegan como "Soporte ProTable · ProTable", de 0:09 y 0:08.

Se usa **Send Audio** porque el nodo de Telegram de n8n **no tiene Send Voice**. Telegram muestra el MP3 en su reproductor, con el título "Soporte ProTable", y no como burbuja de nota de voz. La consigna admite las dos. La alternativa, un HTTP Request a `sendVoice`, obliga a escribir el token del bot en la URL del nodo y lo deja en el JSON exportado: no se hizo. Después del envío, `Set - Fin de flujo (sin binario)` cierra la ejecución sin ningún binario.

Resultado de la ejecución 477: la respuesta llegó al chat como un audio de 9 segundos, y se confirmó en el celular que se escuchaba bien.

Componente 8 · Compliance: el audio no se guarda

Consigna: "los audios se procesan en memoria volátil y se destruyen al cerrar el flujo". Medidas aplicadas, cada una con su límite:

#	Medida	Qué logra	Límite
1	Workflow → Settings: `saveDataSuccessExecution: none` y `saveDataErrorExecution: none`	n8n no guarda las ejecuciones, ni las exitosas ni las fallidas. El audio de entrada, la transcripción y el MP3 no quedan en el historial.	Se pierde la trazabilidad y se depura más difícil.
2	`saveManualExecutions: false` y `saveExecutionProgress: false`	Las pruebas desde el editor y los pasos intermedios tampoco se guardan.	Mientras la pestaña está abierta los datos se ven en el editor: están en memoria del navegador, no persisten.
3	`Set - Entrada de voz (descarta audio)` con **Include Binary File = off**	Desde ese nodo, el audio del cliente **ya no viaja**: no llega al agente, ni a la herramienta, ni a la memoria. El Set final cierra sin binario.	Durante la ejecución, el binario pasa por el almacenamiento temporal de n8n Cloud. Ese modo de almacenamiento es una variable de entorno que en Cloud no se configura.
4	No se registra nada	No hay planilla, Slack ni log con el audio o la transcripción.	La memoria corta guarda el texto de los últimos 6 mensajes del chat mientras dura la sesión.
5	Credenciales en el gestor de n8n	El JSON exportado no lleva ninguna key: las credenciales van por referencia.	—

> **Cuándo se aplicaron los settings.** Los ajustes 1 y 2 se aplicaron el **12/10, después de capturar la evidencia** de este documento. Por eso las ejecuciones de prueba (457 a 479) quedaron en el historial y se pudieron capturar. Desde el cambio, las ejecuciones nuevas no se guardan.

Lo que no se puede destruir desde n8n, y se declara:

- **Telegram** conserva la nota de voz original en el chat y en sus servidores. Es el canal.
- **OpenAI** (Whisper y el modelo del agente) recibe el audio y el texto. **ElevenLabs** recibe solo el texto de la respuesta, no la voz del cliente. Cada uno tiene su propia política de retención. **Control pendiente: verificar las políticas vigentes de los dos proveedores.**

Marco legal. La voz identifica a la persona, y el contenido puede incluir datos del local o de sus clientes: las notas de voz son **datos personales**. En Argentina rige la **Ley 25.326 de Protección de Datos Personales**. El diseño aplica el principio de **minimización**: el audio existe solo mientras corre la ejecución y se usa únicamente para transcribir.

En una frase: el flujo no guarda el audio. Se procesa durante la ejecución, se descarta explícitamente después de transcribirlo y la ejecución no queda en el historial. Lo que retienen Telegram y los proveedores de IA queda fuera del control del flujo, y se declara.

3. Objetivo 1 · Viabilidad: ROI y fatiga cognitiva

Para quién es la voz

Quien pide soporte en ProTable casi nunca está frente a una computadora. Es el encargado de salón en pleno servicio, el jefe de cocina con las manos en la plancha o el dueño que maneja entre dos locales. Para ellos, escribir es caro: frenar, secarse las manos, tipear. Una nota de voz de 8 segundos no lo es. Por eso la voz **complementa** al mail para las consultas urgentes y operativas, no lo reemplaza.

Conviene voz	No conviene voz (mejor texto)
Fallas en servicio: "el pedido sigue trabado en caja", "el QR de la mesa 7 no abre"	Montos, facturas, números de pedido o de comprobante: se escuchan una vez y se olvidan
"¿Cómo hago…?" con respuesta corta	Procedimientos de más de dos pasos (configurar un menú completo, alta de un local)
Usuario con las manos ocupadas o en movimiento	Tablas, comparativas de planes, listados de mesas o productos

Costo por capa (precios de lista consultados el 11/10/2026)

Capa	Proveedor y modelo	Precio
Voz → texto	OpenAI Whisper (`whisper-1`)	**USD 0,006 por minuto** de audio
Razonamiento	OpenAI `gpt-4o-mini` + embeddings de la herramienta	Por tokens, con salida tope de 200. Marginal frente a las otras dos capas. *Precio no verificado para este informe.*
Texto → voz	ElevenLabs Multilingual v2	**USD 0,08 cada 1.000 caracteres** (Flash/Turbo: USD 0,04)

≈ USD 0,0095
Turno real (ejecución 477): 9 s de Whisper (USD 0,0009) + 108 caracteres de TTS (USD 0,0086)

≈ USD 0,019
Peor caso por turno: 30 s de audio (USD 0,003) + 200 caracteres (USD 0,016)

× 5
El TTS cuesta unas 5 veces más que la transcripción: el límite de 200 caracteres es la palanca de costo

Volumen (referencia)	Turnos de voz por mes	Caracteres de TTS	Plan de ElevenLabs que alcanza	Costo de voz aprox.
Piloto (1 local)	50	10.000	Free (no habilita uso comercial)	USD 0,15 (solo Whisper)
10 locales	300	60.000	Starter (USD 6)	≈ USD 7
50 locales	1.300	260.000	Creator (USD 22)	≈ USD 26

Supuesto de la tabla: 200 caracteres por respuesta, el máximo. Los volúmenes son escenarios de referencia, no mediciones.

Dónde está el retorno. No en que la voz sea barata: es más cara que el texto. Está en (1) el **tiempo del cliente en servicio**, que resuelve en una nota de voz lo que le llevaría minutos escribir, justo en el momento de más estrés; (2) el **desvío de llamadas**: una consulta operativa que terminaría en un llamado al equipo se resuelve con el manual por centavos; y (3) un **costo acotado y predecible**, lineal por turno gracias al límite de 200 caracteres y al fallback a texto en errores. **Conclusión:** la voz es viable como canal complementario para consultas operativas cortas. No lo es para facturación ni procesos largos.

Fatiga cognitiva

El audio es **lineal y efímero**: no se puede volver con la vista a la línea anterior, y todo queda en la memoria de trabajo mientras se escucha, con el ruido de un salón encima. Las respuestas de más de 15 o 20 segundos se olvidan desde el principio; las listas habladas obligan a recordar el orden; los números dichos una vez se anotan mal. El límite de 200 caracteres es también una **medida de carga cognitiva**: unos 12 a 15 segundos de audio (la respuesta de la ejecución 477 duró 9 s), una sola idea, y el encargado puede actuar enseguida.

4. Objetivo 3 · Pautas de diseño conversacional para voz (VUI) y privacidad

#	Pauta	Cómo se aplica en el flujo
1	Máximo 200 caracteres	Regla en el System Message y corte determinista en el Code.
2	Una idea por mensaje, primero la acción	"Tocá Refrescar Estado MP en la tarjeta del pedido. Si el pago está aprobado, el pedido pasará a Pendiente."
3	Sin saludos ni despedidas	El prompt pide ir directo a la respuesta: cada segundo de audio cuesta atención y dinero.
4	No leer markdown, links, emojis ni referencias	El Code los limpia antes del TTS, incluidas las citas de sección del manual.
5	No inventar	Solo responde con el manual. Si no está: "No sé, eso no está en el manual. Te paso con una persona del equipo."
6	Pedir aclaración corta	Si no se entiende, pide que lo repita en una frase, en vez de adivinar.
7	Errores amables y accionables, en texto	"Había mucho ruido y no te entendí bien. ¿Me lo repetís en un lugar más tranquilo?". En texto para no gastar TTS.
8	Voz coherente con la marca	Siempre Octavio – Calm Narrator, español, voseo rioplatense en el texto, Stability 0,55.
9	Privacidad por diseño	El audio se usa para transcribir y se descarta. No se registra ni el audio ni la transcripción (Componente 8).

Privacidad de los binarios. Las notas de voz son datos personales (Ley 25.326). El flujo aplica minimización: el audio vive solo durante la ejecución, el Set de entrada lo descarta, las ejecuciones no se guardan y no hay log de contenido. Límites declarados: Telegram conserva el original; OpenAI y ElevenLabs tienen sus propias políticas de retención (control pendiente).

5. Decisiones y desvíos respecto del material de clase y del plan

Punto	Material de clase o plan	Lo construido	Por qué
Transcripción	HTTP Request a Groq (whisper-large-v3)	Nodo nativo OpenAI Whisper	La consigna pide el nodo OpenAI Whisper con `data`.
Texto a voz	HTTP Request a ElevenLabs	Nodo verificado de ElevenLabs	La consigna pide el nodo nativo.
Contingencia	IF de largo mínimo (3 caracteres)	Switch de 5 reglas + salidas de error	Un IF de largo no detecta las alucinaciones de Whisper (ejecución 461).
Truncado	`substring(0, 200)`	Corte por oración o palabra + limpieza	`substring` deja palabras cortadas al final del audio.
Compliance	Un NoOp llamado "Compliance"	Settings reales + Set sin binario	El NoOp documenta, pero no hace nada.
Arquitectura	Plan: duplicar el M5 con los 3 workers como herramientas	Agente de voz propio con la herramienta del manual	Latencia y simplicidad (página 1).
Modelo del agente	Plan: Groq qwen, el del M4	OpenAI gpt-4o-mini	El mismo de los workers del M5, que ya lo habían reemplazado porque el de Groq es un modelo en vista previa.
Log técnico	Plan: planilla con metadatos	Sin registro	Minimización: no se registra nada. Costo: sin trazabilidad de uso.
Respuesta	Send Audio	Send Audio (igual)	El nodo de Telegram de n8n no tiene Send Voice.

6. Pruebas realizadas (ejecuciones reales del 12/10/2026)

Ejecución	Entrada	Resultado	Captura
457, 459, 461	Banco de pruebas sin Telegram (`M6_prueba_voz_sin_telegram`): consulta generada con la voz Helena de Windows (8 s) y 3 s de silencio	**Correcto.** Transcripción: "¡Hola! Un cliente pagó con Mercado Pago y el pedido sigue trabado en caja. ¿Qué tengo que hacer?". Sobre el silencio Whisper alucinó "Subtítulos realizados por la comunidad de Amara.org" y el Switch lo derivó a `alucinacion_ruido`. **Hallazgo:** en 459 y 461 el agente agregaba "Esto está en la sección 14.6 del manual".	C7 (texto)
475	Banco de pruebas, después de sumar la limpieza de citas al Code	**Correcto.** 114 caracteres limpios, MP3 de 145 kB.	—
477	Nota de voz real por Telegram, 8 s, `.oga` de 169 kB: "Hola, un cliente pagó con Mercado Pago y el pedido sigue en caja. ¿Qué hago?"	**Correcto de punta a punta.** El agente consultó el manual. Texto TTS de 108 caracteres: "Tocá "Refrescar Estado MP" en la tarjeta del pedido. Si el pago está aprobado, el pedido pasará a Pendiente." MP3 de 146 kB, Telegram `ok`, audio de 9 s. Se escuchó en el celular.	C0, C2 a C6b
479	Segunda nota real, 8 s, en el mismo chat	**Transcripción robusta:** "Un cliente hizo el pagamento por Mercado Pago, pero el pagamento está pendiente, está pendiente en la caja." Whisper entendió aunque la consulta mezcló portugués. **Hallazgo:** el agente **no volvió a consultar el manual**, se apoyó en la memoria del turno anterior y agregó "Revisá si el cliente eligió Mercado Pago", que no sale del manual. Respuesta TTS de 95 caracteres.	C6b

> **Hallazgo honesto (ejecución 479).** El System Message dice "consultá SIEMPRE la herramienta", pero con la memoria del turno anterior el modelo decidió que ya sabía la respuesta y sumó una recomendación propia. No es una alucinación grave, pero rompe la regla de responder solo con el manual. Dos mejoras posibles: **forzar la consulta en cada turno** (por ejemplo, llamando a la herramienta antes del agente y pasándole los fragmentos como contexto) o **validar la respuesta con el Juez del M8**, que compara lo dicho contra los fragmentos recuperados.

Casos cubiertos por diseño pero no probados de punta a punta: mensaje de texto en vez de audio (salida de error de la descarga), audio de más de 90 s (regla `muy_largo`), audio corrupto (salida de error de Whisper) y falla de ElevenLabs (respuesta en texto). Las ramas existen y están conectadas (C0); falta ejecutarlas con un caso real.

7. Anexo

Ítem	Detalle
Workflow exportado	`checkpoint6_bruno_acorsi.json`, exportado desde n8n. No contiene keys: las credenciales van por referencia (`brunobot`, `OpenAI account`, `Eleven_all`).
Dependencia	Sub-workflow `rag_consultar_manual_modulo5_acorsi_bruno` (entregado en el M5), publicado: n8n no ejecuta sub-workflows sin publicar.
Nodo de la comunidad	`@elevenlabs/n8n-nodes-elevenlabs` (verificado), instalado en la instancia.
Capturas	Ejecución 477 (C0 a C6) y chat de las ejecuciones 477 y 479 (C6b). El identificador del chat está tapado.

Fuentes de precios (consultadas el 11/10/2026). OpenAI, *API Pricing*, tabla *Transcription*: `whisper-1` USD 0,006 por minuto (developers.openai.com/api/docs/pricing). ElevenLabs, *API Pricing*: Multilingual v2/v3 USD 0,08 cada 1.000 caracteres, Flash/Turbo USD 0,04, y planes Free, Starter y Creator (elevenlabs.io/pricing/api). Telegram, *Bot API*: `sendAudio`, `sendVoice` y el límite de 20 MB de `getFile` (core.telegram.org/bots/api). Precio de `gpt-4o-mini`: no verificado para este informe.